

AI-Assisted Testing vs. Automatic Test Generation

Defects4J Csv-1: a four-method case study

CMA-ES | FSCS-ART | KGU Sonnet 5 | KGU Gemini 3.5 Flash Lite

Round 2 submission | 4 October 2026

Abstract

This case study compares four test-generation approaches on one real Defects4J bug. The selected suites contain 30, 30, 21 and 18 JUnit methods. All suites pass the fixed revision twice. Sonnet and Gemini expose a carriage-return line-counting fault on the buggy revision; the two bounded algorithm suites do not. Target-class coverage and recorded timing are reported with their original scopes. Fixed-assisted oracle construction, unequal input domains and a single bug limit the conclusions. This is a measured one-bug study, not a result for the entire Defects4J dataset.

Team members

นายธนินธร อันทรมิตร | 673380043-6

นายศุภกร กรรมรินทร์ | 673380061-4

นายณัฏฐพล เพ็งพล | 673380267-4

นายณัฐกรณ อธิสาร | 673380268-2

Repository: https://github.com/natchapolp-glitch/Project_SQA (branch aom)

1. Study design and measurement

The objective is to build executable unit tests using two implemented algorithms and two university-hosted AI tools, then compare validation, code coverage, fault detection and recorded performance. Csv-1 and `org.apache.commons.csv.ExtendedBufferedReader` are the only experimental case in this submission.

Item	Declared design
Case	Csv-1; one bug, one target class
Methods	CMA-ES, FSCS-ART, KGU Sonnet 5, KGU Gemini 3.5 Flash Lite
Algorithm budget	30 proposed input vectors; seed 101; six method declarations
AI settings	Same prompt/context; temperature 0; maximum output 4096 tokens
Validation	Same suite: fixed1, fixed2, buggy, fixed coverage
Benchmark environment	Defects4J 3.0.1 / Java 11 / America/Los_Angeles
Measurement unit	JUnit method entries, not unique scenarios or number of bugs

Process

```
Source / bounded inputs / prompt
-> four generated JUnit suites
-> fixed revision twice
-> buggy revision with unchanged suites
-> target-class coverage and result table
```

A fault is counted only when a valid suite passes the fixed revision and an assertion on the buggy revision relates to the real patch. Compilation problems, missing dependencies and fixture errors are not counted as faults. Coverage is covered executable lines/branches divided by the measured target-class totals. The two fixed passes validate stability of one suite; they are not independent generation repeats.

2. Algorithm implementations

CMA-ES

The implemented optimizer samples a bounded continuous vector from a multivariate normal distribution. It updates the mean, covariance matrix and step size from ranked candidates, using rank-one/rank-mu covariance updates and cumulative step-size adaptation. Box clamping keeps vectors within $[-1,1]$. The implementation uses Python standard-library arithmetic and a Cholesky factor, with diagonal jitter for robustness. The core strategy follows the CMA-ES principles described by Hansen [1].

For this Java adapter, one vector coordinate selects a method and the remaining coordinates decode arguments/fixtures. The objective minimizes the observed frequency of stable fixed-revision behavior, a proxy for output diversity. It does not optimize line or branch coverage. The budget is 30 proposed vectors, not 30 complete populations; a final partial population is observed without a full update. Initial sigma is 0.30 of mean domain width, and seed101 is recorded.

FSCS-ART

The first point is random. Each later step samples 10 candidates and chooses the candidate whose distance to its nearest selected point is greatest. Euclidean distances are normalized by coordinate-domain width. This implements the max-min spreading principle of adaptive random testing [2]. The Java adapter decodes the selected vectors using the same frozen helper and six-target list as CMA-ES.

JUnit oracle construction

Every proposed vector is executed twice on the fixed revision. Stable, nonduplicate vector/target observations become expected assertions. Fixture failures and unstable outputs remain in the observation ledger instead of being emitted as valid tests. GeneratedStudyTest embeds its required helper. The bounded Reader fixtures use LF-containing strings; they do not include the CR/CRLF sequences that expose the selected fault. Vector diversity therefore does not guarantee semantic input coverage.

Implementation: Algorithm1_CMAES/Code, Algorithm2_FSCSART/Code, Shared/frozen-runtime/scripts/study/generate.py and Shared/inputs/algorithm-targets.json. The bundle also provides a fixed-source regeneration command; newly regenerated results must remain separate from the original reported suites.

3. AI prompting and retained evidence

Both AI tools were accessed through KKU IntelSphere at <https://gen.ai.kku.ac.th>. The requested Sonnet ID is claude-sonnet-5 and the observed response model is anthropic/claude-sonnet-5. Gemini requested/observed gemini-3.5-flash-lite. The source prompt is identical for both methods; its SHA-256 is 485589862422ea6cd0f756145d2e8fef4781232751fffab9ab3c40b6c73c019b.

Setting	Sonnet 5	Gemini 3.5 Flash Lite
Transport	/messages	/chat/completions
Temperature	0	0
Maximum output	4096 tokens	4096 tokens
Thinking	Disabled	Not applicable in retained request
Observed input tokens	63,672	41,314
Observed output tokens	3,636	1,655
Generated/executed methods	21	18

The prompt supplies fixed source and an embedded helper/fixture contract, asks for executable JUnit source, limits the method count and specifies the project/version context. The full exact prompt and settings are included, rather than a reconstructed sample. This makes the study fixed-assisted regression test generation: the AI has access to fixed behavior/source and the result must not be described as blind discovery from buggy-only context.

The selected Java was extracted from the fenced response without editing assertions or pruning failing tests. Raw response text, request intent, model/usage receipts, original extracted sources and packaged suites are included. A prior provider-label check rejected the Messages provider name. Its received Sonnet response was reconciled without resending the request; the failure and no-resend receipt are retained separately.

AI generation is not repeated during the demo. Reproducing an API request may yield different text even with temperature0 and requires university access/quota; replaying the saved suites requires neither API keys nor fresh model tokens.

4. Results on Defects4J

Method	Tests	Fixed failures	Buggy failures	Fault
CMA-ES	30	0 / 0	0	No
FSCS-ART	30	0 / 0	0	No
Sonnet 5	21	0 / 0	1	Yes
Gemini 3.5 Flash Lite	18	0 / 0	1	Yes

The four suites contain 99 method entries in total per stage, not 99 distinct semantic scenarios. Canonical records are the counted Beam Defects4J replay; the Aom replay is supporting evidence and is not an extra generation repeat. All fixed and coverage stages pass. Two valid suites expose the same Csv-1 fault, so the number of distinct bugs in this study remains one.

Method	Lines covered	Line %	Branches covered	Branch %
CMA-ES	31/37	83.78	13/26	50.00
FSCS-ART	31/37	83.78	13/26	50.00
Sonnet 5	36/37	97.30	22/26	84.62
Gemini 3.5 Flash Lite	37/37	100.00	23/26	88.46

Sonnet and Gemini cover more measured target-class lines/branches in this case. Their handwritten-style sequences exercise CR and CRLF behavior outside the bounded algorithm stream domain. These observations support a case-specific explanation; unequal domains and generation effort prevent a controlled superiority claim.

Method	Generation / HTTP seconds	D4J evaluation seconds
CMA-ES	6.77	23.25
FSCS-ART	7.37	22.69
Sonnet 5	26.25	15.24
Gemini 3.5 Flash Lite	6.45	14.88

Algorithm generation time is the original native Windows producer duration, including fixed observations and suite construction. AI time is observed HTTP submit-to-response wall time, not model compute time. D4J time is the canonical evaluation duration; it is not generation time or an end-to-end benchmark. Values are not averaged across hosts or summed into a fair speed ranking.

5. Fault analysis and demo evidence

Csv-1 concerns line counting in `ExtendedBufferedReader` when reading carriage returns and CRLF. The official isolated patch and both benchmark/source representations are included under `Experiment/source-bindings`. AI assertions fail on the buggy revision and pass on the fixed revision, with these concrete examples:

Sonnet: `lineNumberDoesNotDoubleCountCRLF`

```
ExtendedBufferedReader br =
    new ExtendedBufferedReader(new StringReader("A\r\nB"));
br.read(); // A
br.read(); // carriage return
assertEquals(1, br.getLineNumber());
// Csv-1b: expected 1, observed 0
// Csv-1f: passes
```

Gemini: `testCarriageReturnLineNumber`

```
ExtendedBufferedReader br =
    new ExtendedBufferedReader(new StringReader("a\rb\nc"));
br.read(); br.read(); br.read(); br.read();
assertEquals(2, br.getLineNumber());
// Csv-1b: expected 2, observed 1
// Csv-1f: passes
```

CMA-ES and FSCS-ART execute their 30 retained tests successfully on both revisions. That means the sampled suites did not expose this fault, not that the buggy revision is correct. The shared bounded fixture streams omit carriage returns, while the AI suites explicitly test those sequences. Improving algorithm input recipes is future work and is not silently applied to the reported suites.

The demo replays these same saved archives in fresh worktrees, shows fixed/buggy logs, and opens `coverage/summary.csv`. The source names and values above are traceable to the included Java and `failing_tests` files. A fresh rehearsal is kept separately from canonical results, and does not change the number of bugs or generation repeats.

6. Source binding and reproducibility

Generation used frozen v12 code at commit 63ad195623c2ed3f67f3ae232c00c54d3160ce72. The retained native upstream buggy file differs from the isolated bug version reconstructed by Defects4J. The original strict mismatch is retained in the repository; the accepted replay binds the actual benchmark source explicitly before execution. Neither test assertions nor production sources were edited to make a mismatch disappear.

Artifact	SHA-256 / identity
Fixed ExtendedBufferedReader	b09b4e6ae43f1914f8e53e0bad7ba5f9483dc9b3e7ae8889c1d9d40c0de19508
Benchmark buggy reader	4ddf7df1c7c8b9e3d45dc6a3410f802a2e5b641f37089655c3eb2f52cdfd5185
Framework commit	6d54320e0db5a357f9ab38a8e4d2e5ae7e1c09
Target	org.apache.commons.csv.ExtendedBufferedReader

The bundle includes frozen code, the six-target list, original observation ledgers, test archives, raw prompts/responses, settings, source bindings, canonical logs, JUnit XML reports and coverage XML/CSV. dependencies such as Java and the initialized Defects4J distribution must be installed using the official instructions [3]. The included checksums and evidence index identify copied source artifacts.

```
python3 Presentation/replay-demo.py \  
  --d4j /path/to/defects4j/framework/bin/defects4j \  
  --worktrees /path/to/worktrees \  
  --output /fresh/path/csv-demo
```

Use Linux/WSL, Java11 and America/Los_Angeles. Output paths must be new. The script verifies frozen code, suites, project revisions and production-source hashes before using a single CPU slot. It then runs fixed twice, buggy and target coverage. If an environment/source guard fails, keep the failure and report it instead of bypassing the guard.

Algorithm regeneration: run Shared/regenerate-algorithms.py with the same three arguments. It uses only fixed source, the frozen helper, seed101 and budget30; its fresh outputs are separate from reported suites. AI prompts can be reused in KKU by an authorized team member, but a newly generated response requires a separate evaluation record.

7. Limitations and conclusions

Limits of this evidence

Only one bug and one target class are evaluated, with one generated suite per method. No dataset-wide fault detection rate, statistical significance or general model ranking is established. The course document requests Java projects in Defects4J generally; a one-bug submission does not demonstrate all-project completion.

Fixed-assisted source/oracle access makes this a regression-oriented design. Input domains differ: algorithm vectors decode bounded streams, whereas AI generates broader sequences. Budget30 vectors and maximum30 AI methods are not equal CPU/token effort. Coverage measures the class, not the complete project. API generation occurred on a different host/context from benchmark replay; their elapsed times cannot establish a fair performance winner.

The prior 17-project experiment and newer multi-bug development outputs remain in Git history. They are deliberately outside this selected one-bug table, because their models, contexts, processing rules and generation conditions differ. Historical outputs are not relabeled as Sonnet5/Gemini3.5FlashLite or counted as new runs in this submission.

What was learned

Executable tests and high coverage address different questions from real-fault detection. Fixed/buggy replay plus source/patch evidence is needed to interpret a failed assertion. A bounded algorithm can generate valid tests yet miss a fault outside its input recipes. In this Csv case, both AI suites included carriage-return scenarios and exposed the fault; that outcome motivates input-domain improvements and broader controlled evaluation, rather than a universal AI advantage.

สรุปสำหรับทีม

ชุดส่งนี้ทดลอง Csv-1 เพียงหนึ่งบั๊กด้วยวิธี ชุดทดสอบผ่าน fixed ทั้งหมด AI สองตัวพบปัญหานับบรรทัดเมื่ออ่าน carriage return ส่วนอัลกอริทึมไม่พบในขอบเขตอินพุตที่ใช้ ผลและ coverage เป็นของกรณีนี้เท่านั้น ไม่ใช่ผลทั้ง Defects4J ข้อมูลเดิมนยังเก็บอยู่ใน Git โดยไม่รวมยอดเข้าชุดนี้

References and evidence map

- [1] Hansen, N. (2016). The CMA Evolution Strategy: A Tutorial. arXiv:1604.00772. <https://arxiv.org/abs/1604.00772>
- [2] Chen, T. Y., Leung, H., and Mak, I. K. (2004). Adaptive Random Testing. ASIAN2004, LNCS3321, pp.320-329. DOI10.1007/978-3-540-30502-6_23. Author-hosted paper: <https://www.cs.nmsu.edu/~hleung/adaptiveRandomTesting.pdf>
- [3] Defects4J3.0.1. Official framework and reproduction instructions: <https://github.com/rjust/defects4j/tree/v3.0.1>
- [4] Team repository: https://github.com/natchapolp-glitch/Project_SQA. Aom accepted report checkpoint e2ce1e2701e5d08a01cef0481e53ea621bc9956e; frozen generation baseline63ad195623c2ed3f67f3ae232c00c54d3160ce72.
- [5] Course assignment: SQA_Project_2026.pdf, CP353201 Software Quality Assurance, academic year1/2569, requirements1.3-1.10 and deliverables2.2. Repository reference SHA-256: 8bf8f13e6db76f579d69e19f86454aac35f2ba4f231cb0bf3c6b4284aeb7bc91.

What to inspect	Bundle location
Four-method table	Experiment/summary.csv and results.json
CMA-ES / FSCS-ART	Algorithm1_CMAES / Algorithm2_FSCSART
Original AI evidence	AI1_KKU_Claude / AI2_KKU_Gemini
Exact generation code/input	Shared/frozen-runtime and Shared/inputs
Raw benchmark logs/XML	Each method: Result/Defects4J
Source/patch derivation	Experiment/source-bindings
Demo command/rehearsal	Presentation/demo-guide.md and rehearsal
Integrity/provenance	checksums.json; Experiment/evidence-index.json

This document was built from the included canonical records, not from screenshots or invented measurements. The course report, slides, CSV table and demo share this single selected case and method set.